МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

БУДІВНИЦТВА І АРХІТЕКТУРИ

КАФЕДРА КІБЕРБЕЗПЕКИ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

Звіт з лабораторної роботи №3

Дисципліни «Програмування динамічних систем»

Студентки 1-го курсу

групи БІКСм-26

Ненько В. Р.

Перевірив:

Маринський К. В.

Київ-2026

Тема: Розробка графічного інтерфейсу керування роботом (Qt + ROS2).

Мета: Сформувати практичні навички створення графічного інтерфейсу користувача (GUI) у ROS2, навчитися інтегрувати GUI з нодами ROS2, реалізувати передачу команд руху через Topic /cmd_vel, а також організувати відображення статусу системи.

Завдання

1. Підготовка середовища

1.1. Активувати ROS2-середовище.

1.2. Перевірити працездатність команд:

- ros2 node list
- ros2 topic list

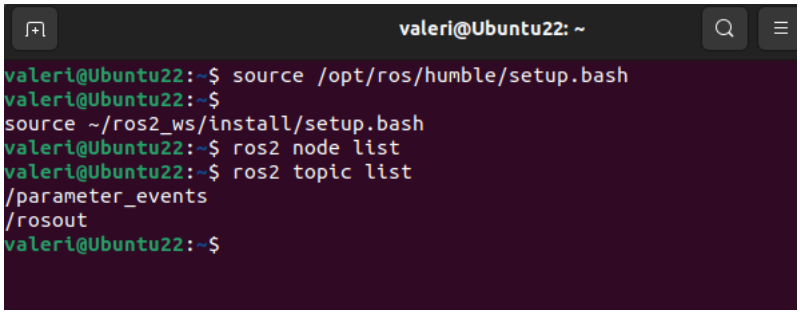


Рис. 1 - Перевірка команд ros2 node list та ros2 topic list після активації середовища ROS2

1.3. Запустити Motion Node з лабораторної №2.

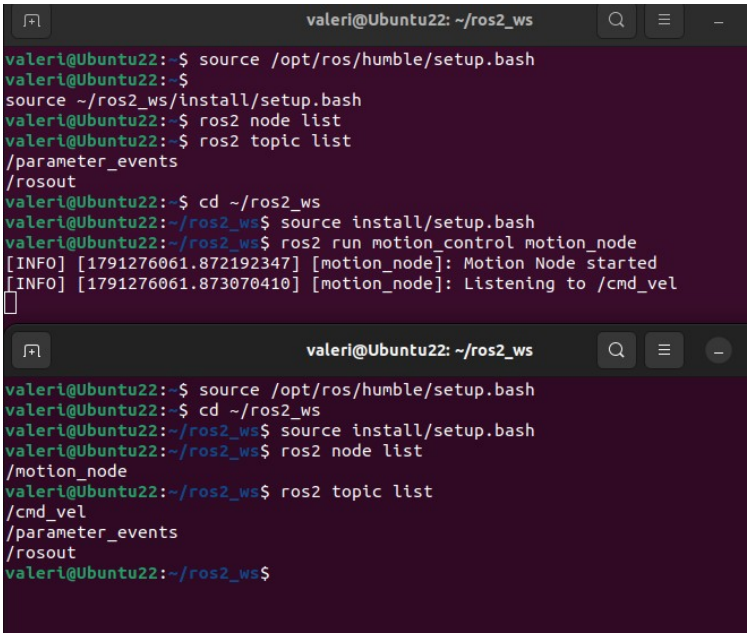


Рис. 2 - Запуск Motion Node з лабораторної роботи №2 (верхній термінал) та перевірка вузлів і топіків у другому терміналі

2. Створення пакету GUI

2.1. Створити новий пакет типу ament_python для реалізації GUI-ноди.

2.2. Додати залежності:

- rclpy
- geometry_msgs

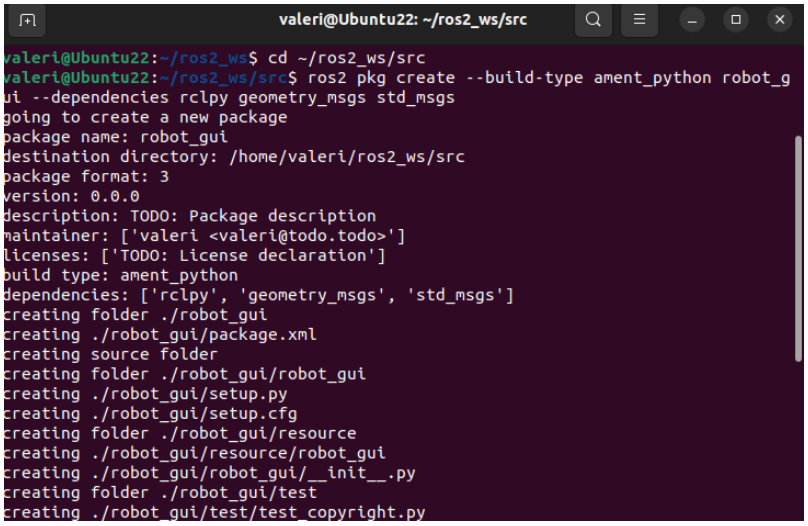


Рис. 3 – Створення нового пакета `robot_gui` типу `ament_python` із залежностями `rclpy`, `geometry_msgs` та `std_msgs`

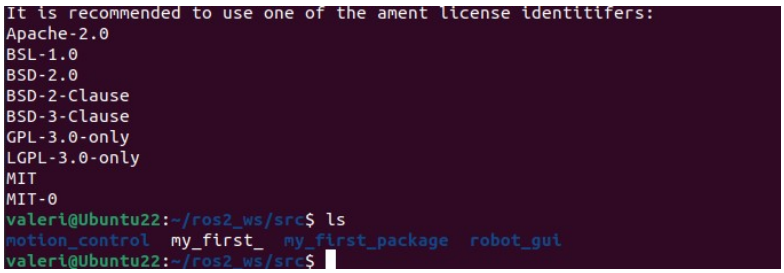


Рис. 4 - Перевірка вмісту каталогу `src` робочого простору після створення пакета

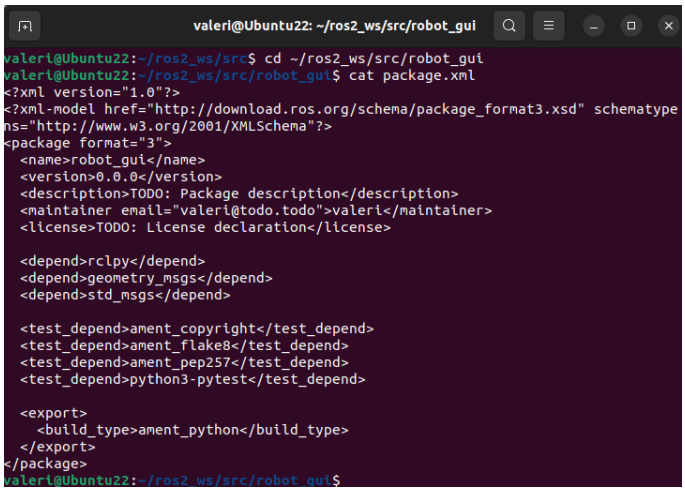


Рис. 5 - Перевірка вмісту `package.xml` та доданих залежностей

2.3. Зібрати workspace та перевірити коректність встановлення пакету.

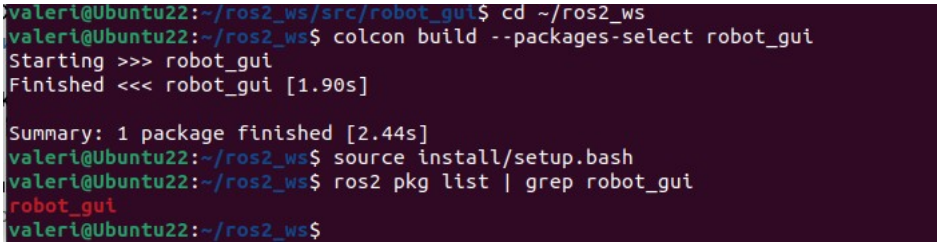


Рис. 6 - Успішна збірка пакета командою `colcon build` та перевірка реєстрації `robot_gui` у ROS2

Залежність std_msgs додано понад мінімальні вимоги завдання, оскільки тип std_msgs/msg/String потрібен для топіка статусу /motion_status.

Перевірка наявності бібліотеки PyQt5:

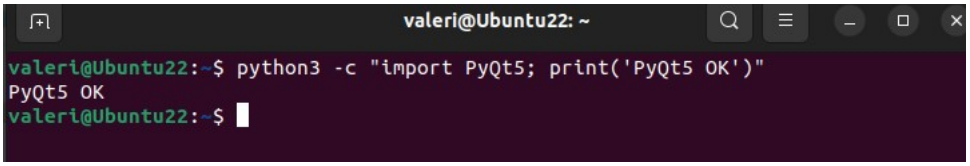


Рис. 7 - Перевірка встановлення PyQt5

3. Розробка GUI

3.1. Створити графічне вікно з кнопками:

- Вперед
- Назад
- Ліворуч
- Праворуч
- Стоп

3.2. Додати текстове поле для відображення статусу.

Файл GUI-ноди створюється в каталозі пакета командою nano (рис. 8). Перша версія містить лише графічну частину без зв’язку з ROS2: імпорти PyQt5 та клас RobotGuiWindow (рис. 9).

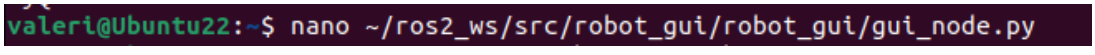


Рис. 8 – Створення файлу gui_node.py

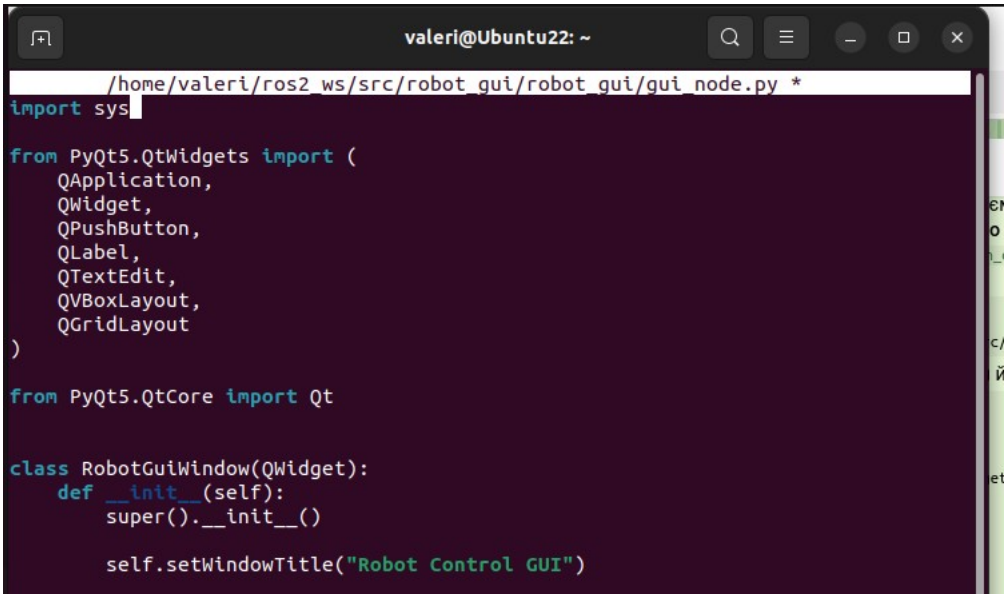


Рис. 9 – Початковий фрагмент коду GUI (імпорти PyQt5)

Після збірки та запуску командою ros2 run robot_gui gui_node відкривається вікно з п’ятьма кнопками та полем статусу (рис. 10). На цьому етапі натискання кнопок лише виводить текст у поле статусу.

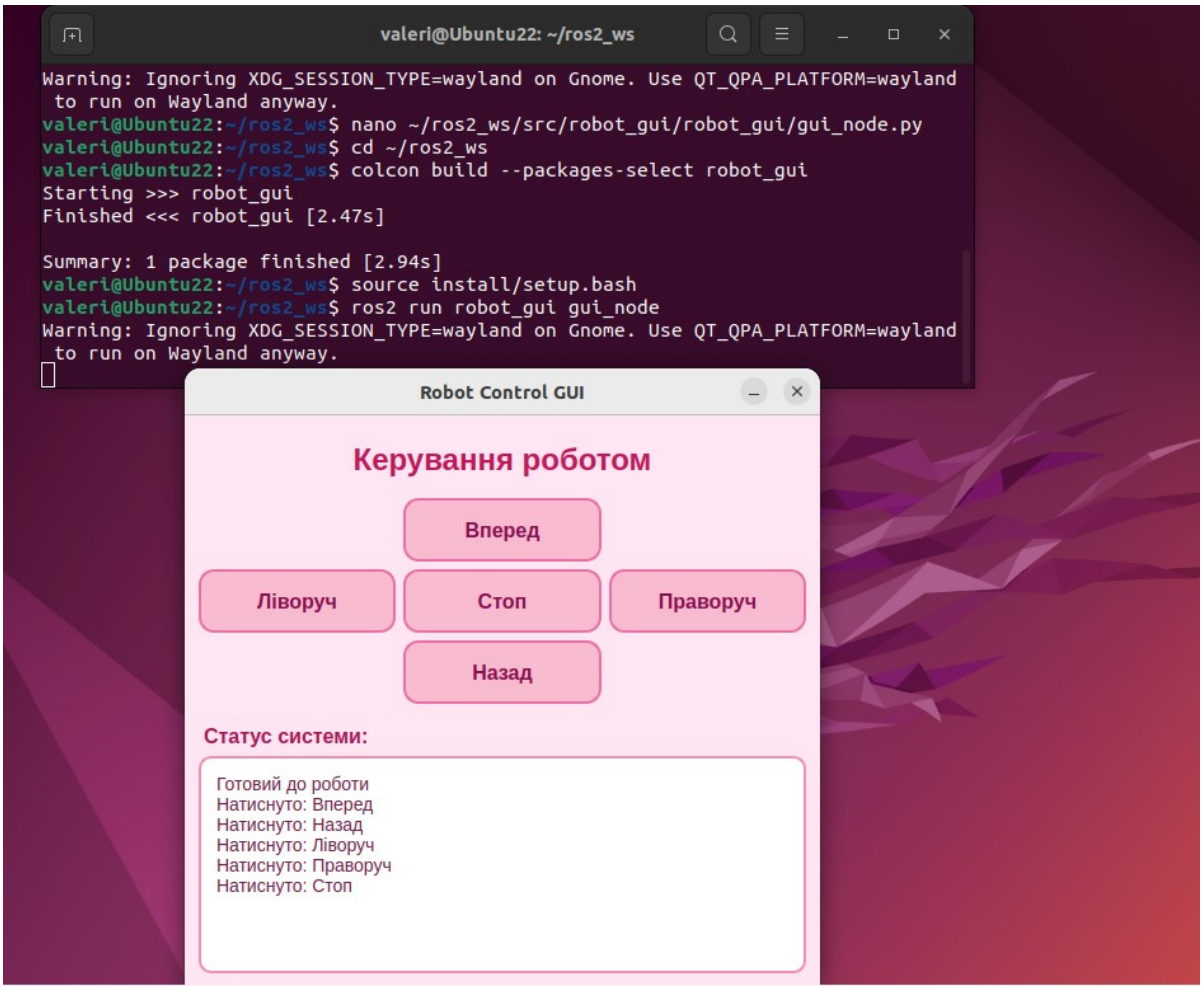


Рис. 10 - Графічне вікно з кнопками та полем статусу (перша версія, без ROS2)

4. Інтеграція GUI з ROS2

4.1. Реалізувати Publisher на Topic: /cmd_vel

4.2. Використати тип повідомлення: geometry_msgs/msg/Twist

4.3. Реалізувати передачу команд при натисканні кнопок:

- linear.x — рух вперед/назад
- angular.z — поворот

До GUI додано Publisher на топік /cmd_vel з типом geometry_msgs/msg/Twist (рис. 11). Кнопки формують повідомлення: «Вперед» і «Назад» змінюють linear.x (+0.5 / −0.5), «Ліворуч» і «Праворуч» змінюють angular.z (+1.0 / −1.0), «Стоп» обнуляє обидва значення (рис. 12). Наявність команд у топіку підтверджено через ros2 topic echo /cmd_vel (рис. 13).



Рис. 11 - Код GUI-ноди після додавання `rclpy`, `Node` та повідомлення `Twist` для інтеграції GUI з ROS2

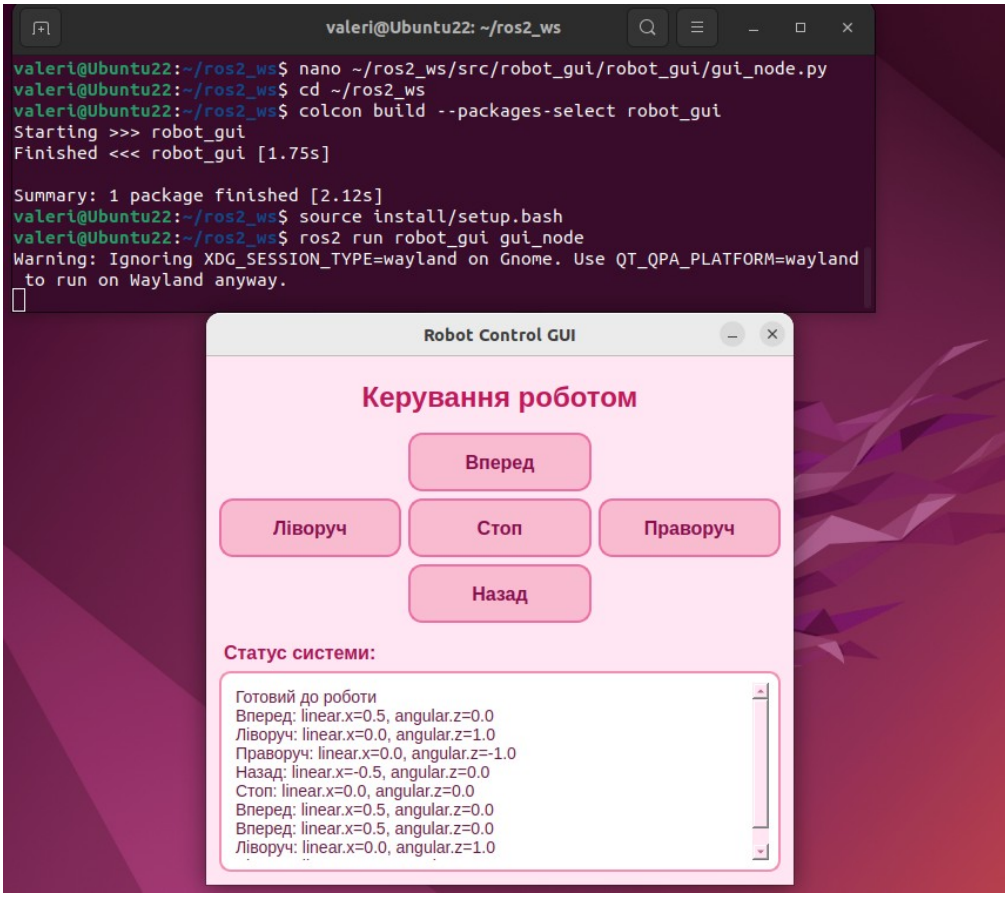


Рис. 12 - Відображення сформованих значень лінійної та кутової швидкості у статусному полі GUI

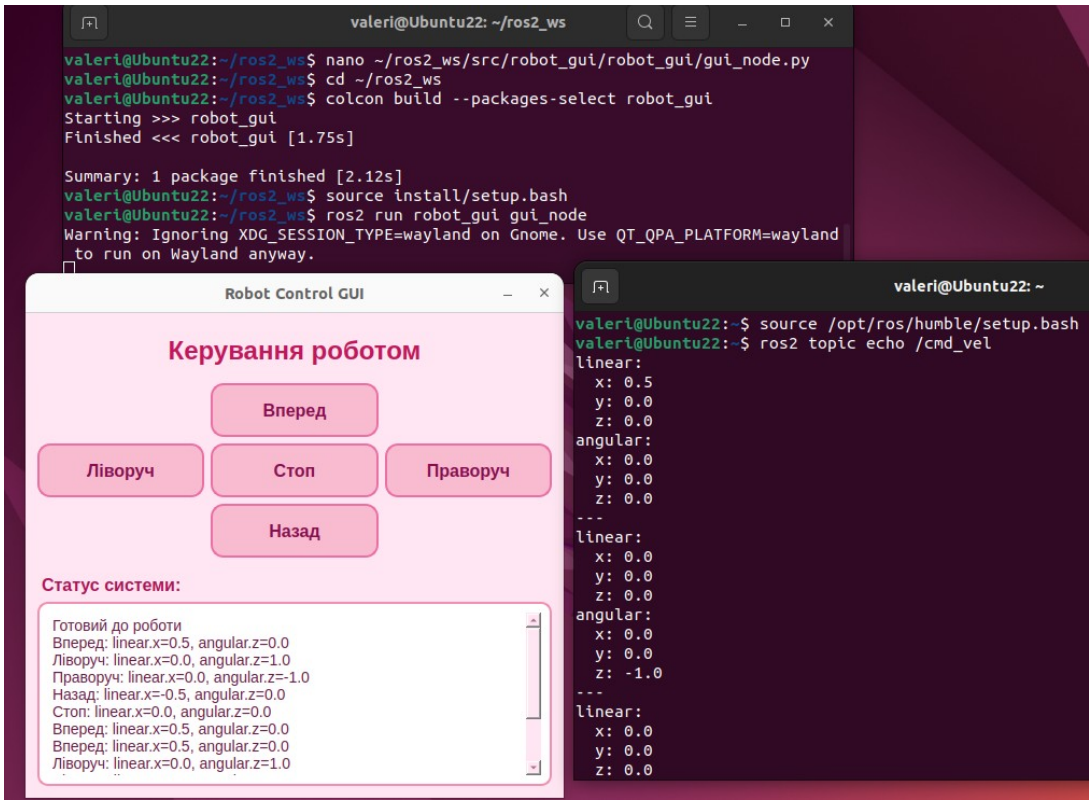


Рис. 13 – Перевірка публікації повідомлень у топік `/cmd_vel` за допомогою утиліти `ros2 topic echo`

5. Відображення статусу

5.1. Реалізувати підписку на Topic статусу (наприклад /motion_status).

5.2. Виводити отримані повідомлення у GUI.

GUI підписується на топік /motion_status (std_msgs/msg/String); отримані повідомлення додаються в поле статусу з префіксом «[Статус]». Для перевірки підписки в топік вручну надіслано тестове повідомлення командою ros2 topic pub --once; воно з’явилося в GUI (рис. 14).

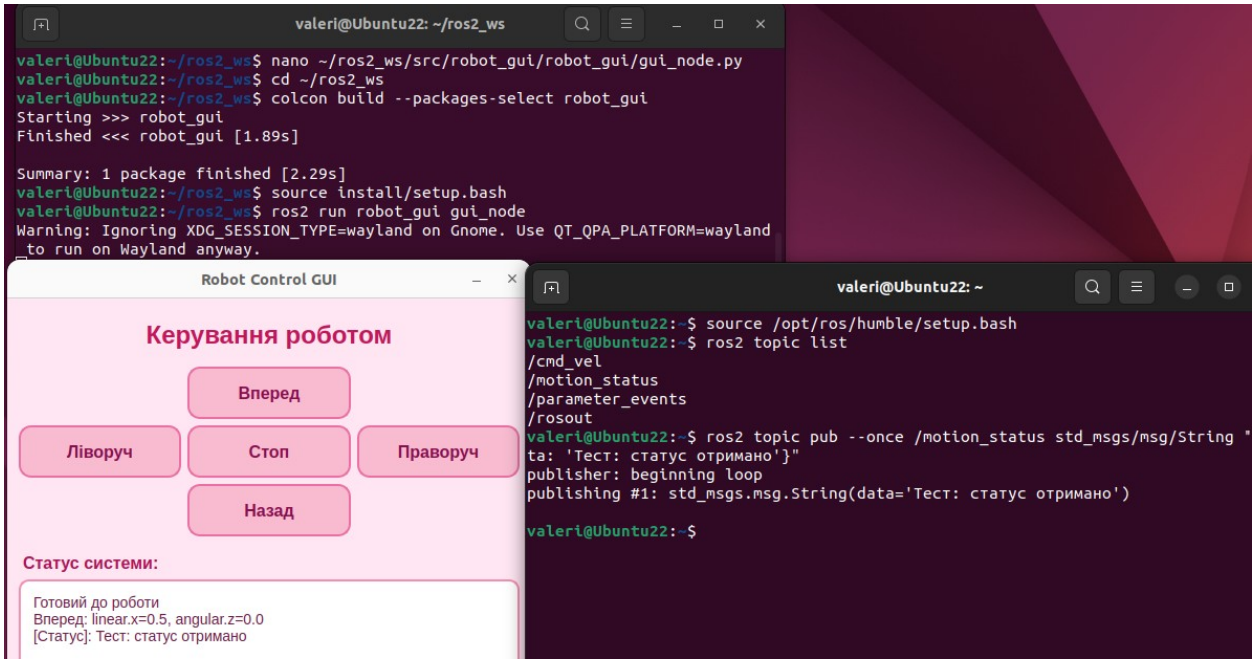


Рис. 14 – Тестування прийому вхідних повідомлень підписником GUI командою ros2 topic pub через тестову публікацію в топік `/motion_status`

6. Тестування

6.1. Запустити GUI-ноду.

GUI-ноду запущено командою ros2 run robot_gui gui_node (див. рис. 10 та рис. 12).

6.2. Запустити Motion Node.

Додавання публікації статусу в Motion Node

У `Motion Node` додаємо **Publisher для Topic** `/motion_status` та повідомлення типу `std_msgs/String`.

Це потрібно для того, щоб `Motion Node` після отримання команди з `/cmd_vel` визначав поточний стан робота (рух вперед, назад, поворот або зупинка) та передавав цей статус у GUI.

Таким чином, організовується зворотний зв’язок між Motion Node та графічним інтерфейсом:

`GUI → /cmd_vel → Motion Node → /motion_status → GUI`

При цьому існуюча логіка обмеження швидкості `Motion Node` залишається без змін.

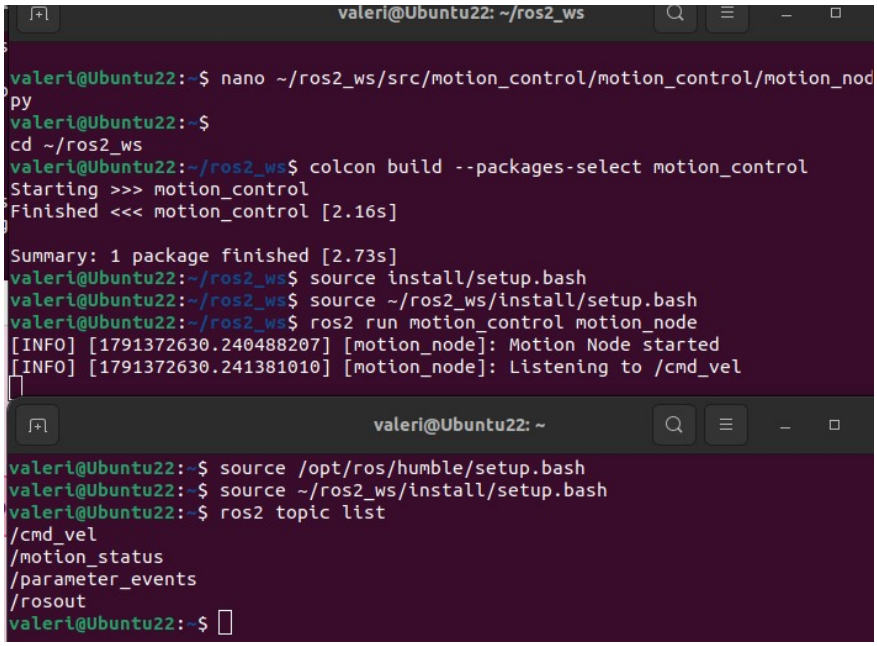


Рис. 15 - Перебудова та запуск Motion Node з Publisher /motion_status; новий топік у ros2 topic list

Перезбірка пакета `motion_control`, запуск `motion_node` із підтримкою публікації статусу та перевірка топіків

6.3. Перевірити керування роботом через GUI.

GUI, Motion Node та ros2 topic echo /motion_status працюють одночасно: після кожної команди в полі GUI з’являється рядок «[Статус]: …», який збігається з виводом echo (рис. 16).

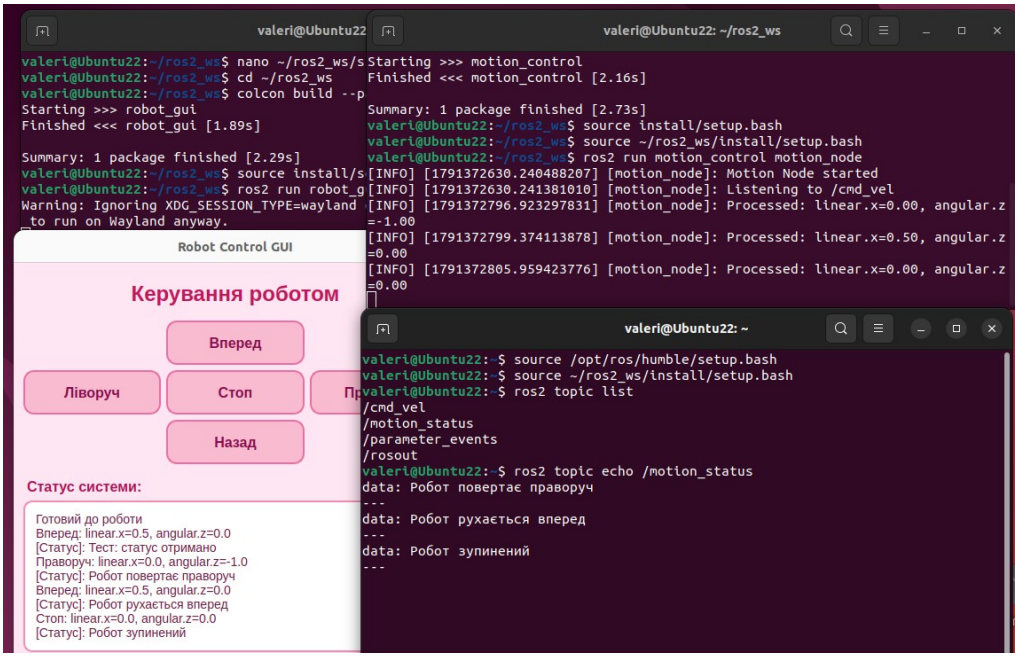


Рис. 16 – Спільна робота GUI та Motion Node: статуси руху в полі GUI та в ros2 topic echo /motion_status

6.4. Переконатися, що команди передаються коректно.

Команди, що передаються з GUI, контролюються в топіку `/cmd_vel` (рис. 17), а `Motion Node` у своєму логу показує оброблені значення `linear.x` та `angular.z` (рис. 16).

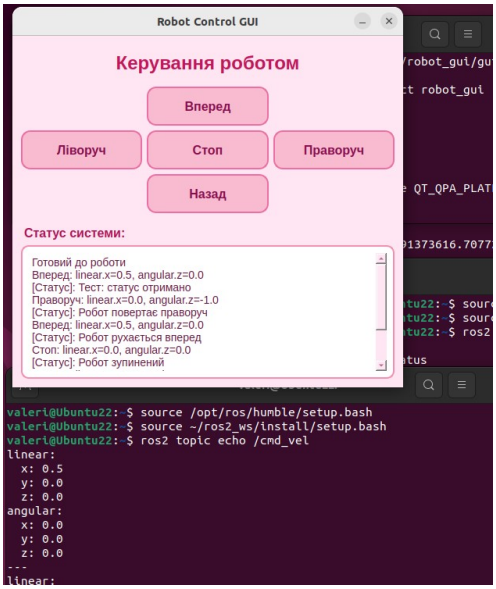


Рис. 17 – Контроль команд: `ros2 topic echo /cmd_vel` та відповідні статуси в GUI

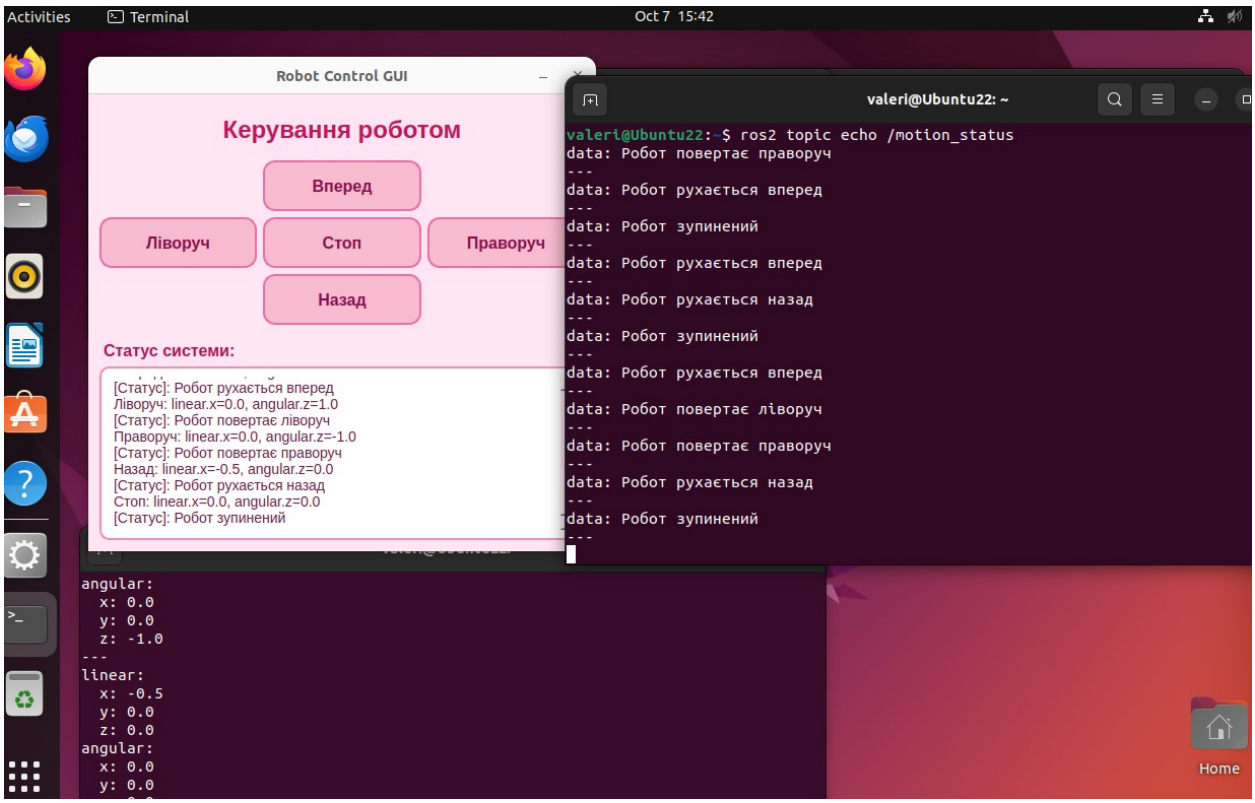


Рис. 18 – Перевірка всіх п'яти кнопок: команди в GUI, статуси в полі GUI та в `ros2 topic echo /motion_status`

Опис реалізації GUI-ноди

GUI-ноду реалізовано в пакеті `robot_gui (ament_python)` у файлі `gui_node.py` з використанням PyQt5. У функції `main()` ініціалізується `rclpy`, створюється вузол `robot_gui_node` та застосунок QApplication. Клас `RobotGuiWindow(QWidget)` отримує цей вузол, створює `Publisher` і `Subscriber` та будує інтерфейс: заголовок, п’ять кнопок («Вперед», «Назад», «Ліворуч», «Праворуч», «Стоп»), та текстове поле `QTextEdit` (лише для читання) для відображення статусу.

Щоб цикл подій Qt не блокувався `rclpy.spin()`, використано `QTimer`: кожні 20 мс викликається `rclpy.spin_once(node, timeout_sec=0)`, що дозволяє обробляти `callback` підписки без зависання вікна.

Покрокові зміни коду: що і куди вписується

Щоб не переписувати файл щоразу заново, код розвивався малими кроками. Нижче наведено, що саме знайти у файлі та що додати або замінити. Проміжні версії файлу та патчі (diff) наведено в репозиторії.

Етап 1. GUI без ROS2 (рис. 8–10).

Створюється файл `gui_node.py` з класом `RobotGuiWindow`, п’ятьма кнопками та полем статусу; кнопки лише дописують «Натиснуто: …» у поле статусу.

Етап 2. Додавання Publisher /cmd_vel (рис. 11–13).

1) Імпорти. Після import sys додати:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
```

2) Конструктор. Замінити def __init__(self): на:

```
def __init__(self, ros_node):
    super().__init__()
    self.ros_node = ros_node
    self.cmd_vel_publisher = self.ros_node.create_publisher(
        Twist, '/cmd_vel', 10)
```

3) Кнопки. Замінити `update_status`("Натиснуто: …") у п’яти підключеннях на `send_velocity`(0.5, 0.0, "Вперед"), (−0.5, 0.0, "Назад"), (0.0, 1.0, "Ліворуч"), (0.0, −1.0, "Праворуч"), (0.0, 0.0, "Стоп").

4) Метод `send_velocity`. Перед `update_status` додати метод, що створює `Twist`, заповнює `linear.x` та `angular.z` і публікує повідомлення. 5) У `main()` створити вузол через `rclpy.init() i Node("robot_gui_node")` та передати його у вікно.

Етап 3. Додавання Subscriber /motion_status (рис. 14).

1) Після імпорту `Twist` додати: `from std_msgs.msg import String`.

2) Знайти `from PyQt5.QtCore import Qt` і замінити на `from PyQt5.QtCore import Qt, QTimer`.

3) У `__init__` після блоку створення `Publisher` додати підписку:

```
self.status_subscriber = self.ros_node.create_subscription(
    String, '/motion_status', self.status_callback, 10)
```

4) У `__init__` після `self.create_gui()` додати таймер (`Qt i ROS2` мають окремі цикли подій, тому `rclpy.spin()` заблокував би вікно):

```
self.ros_timer = QTimer(self)
self.ros_timer.timeout.connect(self.check_ros_events)
self.ros_timer.start(20)
```

5) Перед `update_status` додати методи `status_callback` (виводить «[Статус]: …» у поле статусу) та `check_ros_events` (`викликає rclpy.spin_once(self.ros_node, timeout_sec=0)`).

Етап 4. Зміни в Motion Node (рис. 15).

У файлі `motion_node.py` (ЛБ №2):

1) додати `from std_msgs.msg import String`;

2) у `__init__` після підписки на `/cmd_vel` додати `self.status_publisher = self.create_publisher(String, '/motion_status', 10);`

3) у кінець `cmd_vel_callback`, після запису `Processed` у лог, додати формування тексту статусу (вперед / назад / ліворуч / праворуч / зупинений) та його публікацію. Логіку обмеження швидкості не змінено.

Фрагменти коду

Створення GUI:

```
self.forward_button = QPushButton("Вперед")
self.backward_button = QPushButton("Назад")
self.left_button = QPushButton("Ліворуч")
self.right_button = QPushButton("Праворуч")
self.stop_button = QPushButton("Стоп")


button_layout = QGridLayout()
button_layout.addWidget(self.forward_button, 0, 1)
button_layout.addWidget(self.left_button, 1, 0)
button_layout.addWidget(self.stop_button, 1, 1)
button_layout.addWidget(self.right_button, 1, 2)
button_layout.addWidget(self.backward_button, 2, 1)


self.status_box = QTextEdit()
self.status_box.setReadOnly(True)
```

Publisher /cmd_vel та відправлення команд:

```
self.cmd_vel_publisher = self.ros_node.create_publisher(
    Twist, '/cmd_vel', 10
)


self.forward_button.clicked.connect(
    lambda: self.send_velocity(0.5, 0.0, "Вперед"))


def send_velocity(self, linear_x, angular_z, direction):
    msg = Twist()
    msg.linear.x = linear_x
    msg.angular.z = angular_z
    self.cmd_vel_publisher.publish(msg)
    self.update_status(
        f"{direction}: linear.x={linear_x}, angular.z={angular_z}")
```

Підписка на /motion_status та обробка подій ROS2:

```
self.status_subscriber = self.ros_node.create_subscription(
    String, '/motion_status', self.status_callback, 10
)


self.ros_timer = QTimer(self)
self.ros_timer.timeout.connect(self.check_ros_events)
self.ros_timer.start(20)


def status_callback(self, msg):
    self.update_status(f"[Статус]: {msg.data}")


def check_ros_events(self):
    rclpy.spin_once(self.ros_node, timeout_sec=0)
```

Публікація статусу в Motion Node (фрагмент):

```
self.status_publisher = self.create_publisher(String, '/motion_status', 10)


if linear_x > 0:
    status_text = 'Робот рухається вперед'
elif linear_x < 0:
    status_text = 'Робот рухається назад'
elif angular_z > 0:
    status_text = 'Робот повертає ліворуч'
elif angular_z < 0:
    status_text = 'Робот повертає праворуч'
else:
    status_text = 'Робот зупинений'


status_msg = String()
status_msg.data = status_text
self.status_publisher.publish(status_msg)
```

Опис взаємодії: GUI + ROS2 + робот

Користувач натискає кнопку в GUI. Обробник `send_velocity()` формує повідомлення `geometry_msgs/msg/Twist` (`linear.x` для руху вперед/назад, `angular.z` для повороту) і публікує його в топік `/cmd_vel`. `Motion Node` підписаний на `/cmd_vel`, у `callback` обмежує швидкості значенням 1.0 (м/с та рад/с), виводить оброблені значення в лог (імітація керування роботом) і визначає поточний стан руху. Цей стан публікується як `std_msgs/msg/String` у топік `/motion_status`. GUI, підписаний на `/motion_status`, отримує повідомлення й виводить його в поле статусу.

Висновки

Сформувала практичні навички створення графічного інтерфейсу користувача (GUI) у ROS2, навчилася інтегрувати GUI з нодами ROS2, реалізувала передачу команд руху через `Topic /cmd_vel`, а також організувала відображення статусу системи

У ході роботи створено пакет `robot_gui` з графічним інтерфейсом на `PyQt5`, який публікує команди руху у топік `/cmd_vel` (тип `Twist`) і відображає статус із топіка `/motion_status`. Роботу GUI перевірено разом із `Motion Node` з лабораторної роботи №2; команди та статуси збігаються у GUI, у `ros2 topic echo` і в логах вузла.

Порівняно з консольним керуванням `(ros2 topic pub)`, GUI зручніший: не потрібно вводити довгі команди з вкладеною структурою `Twist`, менше ймовірність помилок, а поточний статус та надіслані значення видно в одному вікні. Недоліки поточної реалізації: залежність від графічного середовища та бібліотеки `PyQt5`. Для налагодження та детального аналізу повідомлень консольні інструменти (`ros2 topic echo`) залишаються корисними, тому на практиці GUI та консоль доповнюють одне одного.